

OPENTELEMETRYを活用したCLAUDE CODEのOBSERVABILITY検証

検証日: 2026-06-25 | 対象: Claude Code v2.1.179

まとめ

- OpenTelemetry (OTel) Collector
- Prometheus
- Loki
- Tempo
- 構成: • Grafana
- 取得できる: ユーザ入力本文・モデル出力本文・ツール入出力・分散トレース
- 取得できない: 拡張思考 (extended thinking) 本文



背景と目的

背景

- Claude Code はシェル実行・ファイル編集・MCP 連携など**広範な操作権限**を持つエージェントである。
- 組織利用では「誰が・何を入力し・どのツールを・どう承認して実行し・何が出来たか」を 事後に追跡できること(=監査証跡)が、要求される。

目的

- Claude Code の OTel 出力から**実際に取得できる監査情報の範囲を実機検証**する。
- 取得に伴うプライバシー上の制御点と限界を整理する。

- OpenTelemetry (OTel) は観測データを送出するための標準規格。
- 信号は3種に大別される。

信号	定義	監査での意味	本PoCの保存先
メトリクス	数値の時系列	「いつ・どれだけ」使ったか(量)	Prometheus
ログ/イベント	個々の出来事の記録	「何が起きたか」(入力・ツール実行等)	Loki
トレース	1リクエストの処理経路と所要時間	「どの順序で処理されたか・どこが遅いか」	Tempo

- Claude Code は環境変数の設定により、これら3信号を OTLP (gRPC/HTTP) で出力する機能を標準で備える。
- 本検証ではこの出力を手元の Collector で受信する。

3. ARCHITECTURE システム構成

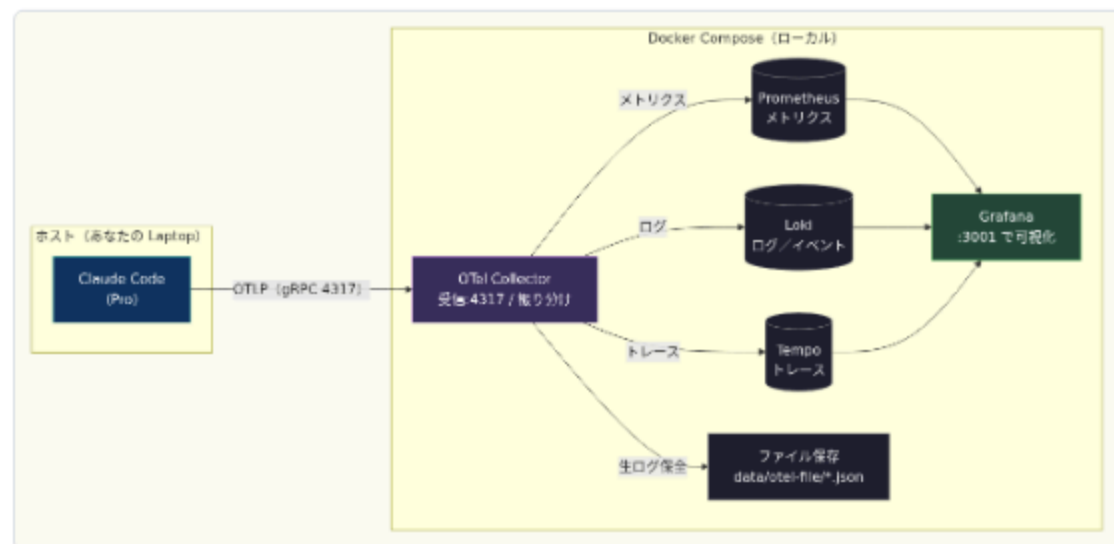


図1. データフロー。ホストの Claude Code が OTLP(gRPC:4317)で送信し、Collector が信号ごとに振り分ける。

要素	役割
OTel Collector	OTLP 受信・振り分け・ファイル保全
Prometheus	メトリクス蓄積(スクレイプ)
Loki	ログ/イベント蓄積(OTLP push)
Tempo	トレース蓄積
Grafana	横断可視化・相関
file exporter	生 JSON をローカル保全

受信側はすべて Docker Compose。ホストは `claude` に環境変数を渡して起動するのみ。

受信側: OTEL COLLECTOR の設定

対象ファイル: 本リポジトリの `otel/config.yaml` (Collector コンテナにマウント)。
Collector は受信(receivers)→加工(processors)→送信 exporters) という流れ(pipeline)を信号ごとに定義する。本PoCは「OTLPで受信し、各信号を複数の送信先へ同時に流す」だけで加工は行わない。

```
# otel/config.yaml
receivers: # 受け口: Claude Code からの OTLP を受信
  otlp: { protocols: { grpc: { endpoint: 0.0.0.0:4317 } } }
exporters: # 送り先
  prometheus: { endpoint: 0.0.0.0:8889 } # メトリクス
  otlphttp/loki: { endpoint: http://loki:3100/otlp } # ログ/イベント
  otlp/tempo: { endpoint: tempo:4317 } # トレース
  file/logs: { path: /data/otel-file/logs.json } # ローカル保存
service:
  pipelines: # 信号ごとに「受信→送信」の経路を定義
    metrics: { receivers: [otlp], exporters: [prometheus, file/metrics] }
    logs: { receivers: [otlp], exporters: [otlphttp/loki, file/logs] }
    traces: { receivers: [otlp], exporters: [otlp/tempo, file/traces] }
```

送信側: CLAUDE CODE の恒常設定(組織配布)

対象ファイル: Claude Code の管理者設定ファイル `managed-settings.json` (Linux/WSL: `/etc/claude-code/`、macOS: `/Library/Application Support/ClaudeCode/`、Windows: `C:\ProgramData\ClaudeCode\`)。 `env` ブロックに記述すると全ユーザへ一括適用され、MDM等で配布でき、ユーザ側では上書きできない。

```
// /etc/claude-code/managed-settings.json
{
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1",
    "OTEL_METRICS_EXPORTER": "otlp",
    "OTEL_LOGS_EXPORTER": "otlp",
    "OTEL_EXPORTER_OTLP_PROTOCOL": "grpc",
    "OTEL_EXPORTER_OTLP_ENDPOINT": "http://collector.example.com:4317"
  }
}
```

出典: 公式ドキュメント「管理者設定」
code.claude.com/docs/ja/monitoring-usage#administrator-configuration

取得粒度の制御パラメータ(プライバシー設計)

既定では機微情報はマスクされる。以下の環境変数を有効化すると、取得内容が段階的に拡張される。

環境変数	有効化で取得される情報	既定
<code>OTEL_LOG_USER_PROMPTS</code>	ユーザ入力の本文(<code>prompt</code>)	OFF
<code>OTEL_LOG_TOOL_DETAILS</code>	ツール入力引数(bash コマンド・対象パス・MCP名 等)	OFF
<code>OTEL_LOG_TOOL_CONTENT</code>	ツール入出力の内容(トレース必須・60KB 切詰)	OFF
<code>OTEL_LOG_RAW_API_BODIES</code>	Messages API の request/response 全文(=会話履歴・モデル出力本文)	OFF
<code>CLAUDE_CODE_ENHANCED_TELEMETRY_BETA</code> + <code>OTEL_TRACES_EXPORTER</code>	分散トレース(span)の出力(ベータ)	OFF

注: `OTEL_LOG_RAW_API_BODIES` は会話履歴の全文を含むため最も機微である。公式ドキュメントでも、これを有効化することは他の各環境変数が開示する情報すべてを含めることに同意する意味だと明記されている。なお `=file:<dir>` を指定すると、60KB で切り詰めずに全文をファイルへ保存できる。

出典: 公式ドキュメント「設定の詳細 > 一般的な設定変数」および「トレース(ベータ)」
code.claude.com/docs/ja/monitoring-usage#common-configuration-variables / #traces-beta



検証方法

手順

1. 各サービス(Collector / Prometheus / Loki / Tempo / Grafana)のコンテナを起動し、起動完了(疎通)を確認する。
2. テレメトリを有効化した状態(§ 4・§ 5 の環境変数を付けた状態)で Claude Code を起動し、自動実行モードでツールを使うプロンプト(ファイル作成・bash 実行など)を1回与える。
3. § 5 の環境変数をすべて有効化し、トレースも有効化した「取得範囲を最大化した状態」でも同様に実行する。
4. 送信されたデータが各保存先(Prometheus / Loki / Tempo)とローカル保存ファイルに**実際に届いているか**を確認し、どの**メトリクス**(数値項目)・**イベント**(記録された出来事)・**属性**(各記録に付くキー値、例: `user.email`)・**トレース**(処理の親子構造)が取得できたかを実データで点検する。

補足: 各イベントには、同じプロンプトに由来する記録を束ねる識別子 `prompt.id` が付く。これを用いて、1つの入力に対する一連の記録(入力 → API 呼び出し → ツール実行 → 出力)が欠けなく時系列でつながることを確認した。

検証環境(バージョン)

Claude Code **2.1.179** / otel-collector-contrib **0.115.1** / Prometheus **2.55.1** / Loki **3.3.2** / Tempo **2.6.1** / Grafana **11.4.0**。ホスト: WSL2 (Linux)。



結果 (1) メトリクス

メトリクス(Prometheus 名)	意味
<code>claude_code_token_usage_tokens_total</code>	トークン数(type: input/output/cacheRead/cacheCreation)
<code>claude_code_cost_usage_USD_total</code>	推定コスト(model 別)
<code>claude_code_session_count_total</code>	セッション数
<code>claude_code_lines_of_code_count_total</code>	変更行数(added/removed)
<code>claude_code_code_edit_tool_decision_total</code>	編集ツールの許可/拒否
<code>claude_code_active_time_seconds_total</code>	実利用時間

ラベルに `model` / `session_id` / `user_email` / `organization_id` / `terminal_type` / `service_version` 等を保持。

コストは API 単価換算の推定値であり、定額プラン(Pro/Max)では実課金されない(利用量の目安)。

出典: 公式ドキュメント「メトリクス」「メトリクスの詳細」「標準属性」

code.claude.com/docs/ja/monitoring-usage#metrics / [#metric-details](#) / [#standard-attributes](#)

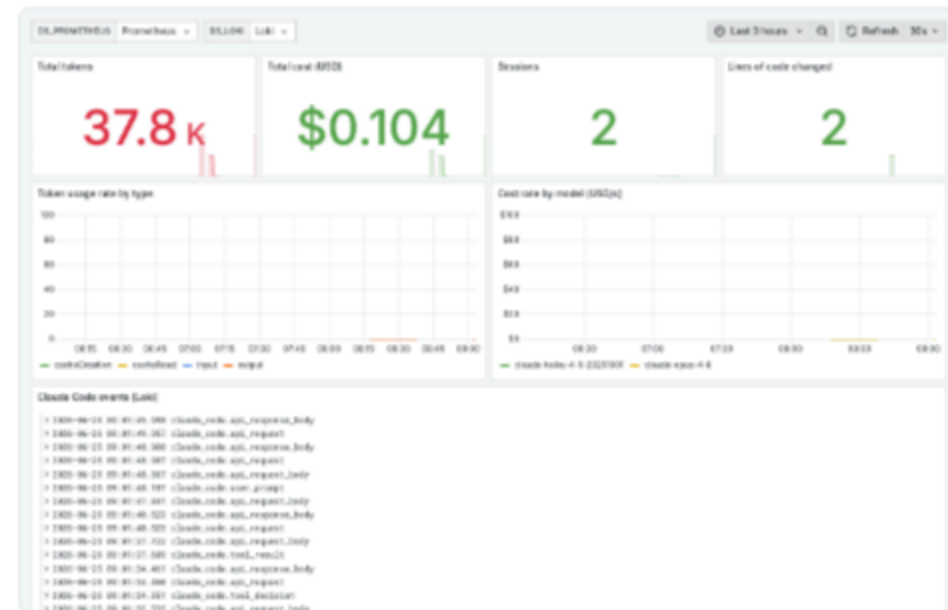


図2. Grafana ダッシュボード(実測値で描画)。累計トークン・推定コスト・セッション数とトークン種別/モデル別の推移。

6. RESULTS

結果 (2) イベント / ログ

Claude Code の「ログ」信号は、自由記述のテキストログ行ではなく、`event.name` を持つ **構造化イベント**として OTLP ログで出力される(=本資料の「ログ」はこのイベント群を指す)。
Loki では `{service_name="claude-code"} | event_name="_"` で絞り込める。主要イベントと監査上重要な属性(実測)を示す。

イベント (<code>event.name</code>)	主な属性	監査観点
<code>user_prompt</code>	<code>prompt_length</code> / <code>prompt</code> (本文※)	ユーザ入力
<code>api_request</code>	<code>model</code> , <code>input/output/cache tokens</code> , <code>cost_usd</code> , <code>duration_ms</code>	トークン・コスト
<code>api_response_body</code> ※	<code>body</code> (応答 JSON = モデル出力本文)	Claude 出力
<code>tool_result</code>	<code>tool_name</code> , <code>success</code> , <code>duration_ms</code> , <code>tool_input</code> (引数※)	ツール実行
<code>tool_decision</code>	<code>tool_name</code> , <code>decision(accept/reject)</code> , <code>source</code>	権限判断
<code>api_error</code> / <code>api_refusal</code>	<code>status_code</code> , <code>error</code> , <code>stop_reason</code>	失敗・拒否
他	<code>auth</code> , <code>mcp_server_connection</code> , <code>permission_mode_changed</code> , <code>plugin_loaded</code> , <code>skill_activated</code> , <code>at_mention</code> ...	

全イベント共通: `session.id` / `user.id` / `user.email` / `organization.id` / `terminal.type` / `event.timestamp` / `prompt.id` 。 (※ は §5 のフラグ有効時のみ。)

出典: 公式ドキュメント「イベント」(各イベント定義・属性)

code.claude.com/docs/ja/monitoring-usage#events / [#user-prompt-event](#) / [#tool-result-event](#) / [#api-response-body-event](#)



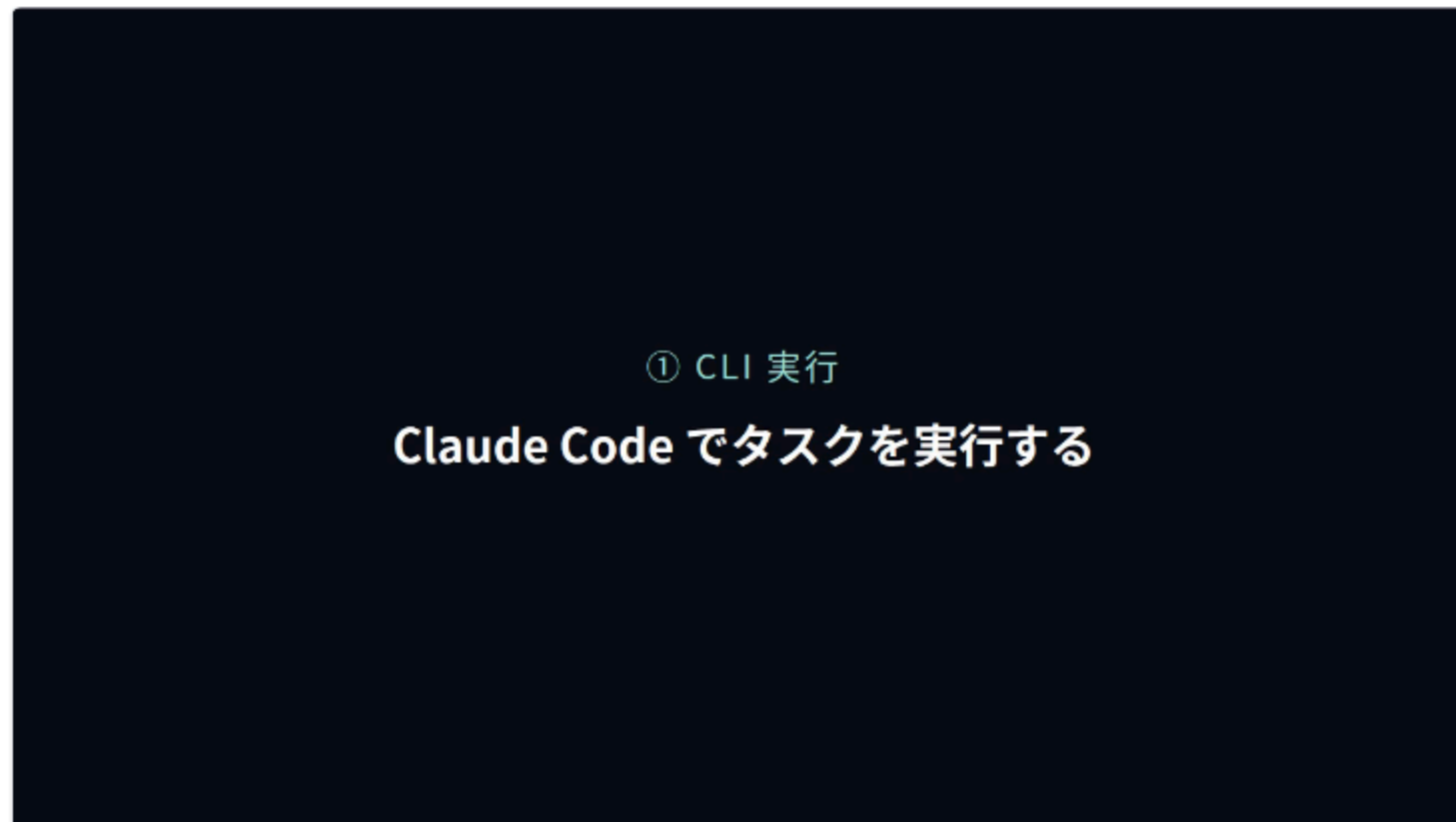


図3. 統合デモ(動画)。①実機の Claude Code CLI でタスクを実行(Write/Read/Bash の実行が画面に出る) → ②結果1 メトリクス → ③結果2 ログ → ④結果3 トレース。各章で1回の実行を構成する操作(①入力 → ②Write → ③Read → ④Bash → ⑤応答)が、どのメトリクス・ログ・トレースに対応するかを日本語で明示。※ メール・組織ID・各種 UUID はマスク処理済み。

対応関係(要約): ①入力=user_prompt / ②③④ツール=tool_result(Write・Read・Bash)・tool_decision / ⑤応答=api_request・api_response_body・token/cost メトリクス / トレースは interaction → llm_request・tool-execution。補足動画: 同一時系列の左右対比 demo/side-by-side.mp4、Grafana 各画面の詳細 demo/gui-demo.mp4 + presentation/gui-guide.md。

6. RESULTS

結果 (3) 分散トレース

分散トレースとは、1件の処理(ここでは「ユーザの1プロンプト処理」)が内部で行った個々の処理を、**span**(=処理区間。名前・開始/終了時刻・親子関係をもつ最小単位)の集まりとして記録したもの。span 同士の親子関係を **span 階層**と呼ぶ。

Claude Code のトレース出力は現在ベータ提供で、§5 の **CLAUDE_CODE_ENHANCED_TELEMETRY_BETA** と **OTEL_TRACES_EXPORTER** を設定すると出力される。本検証ではこれを有効化して実行し、生成された span が **Tempo** に送信・保存され、**Grafana** で参照できることを確認した。

SPAN 階層(実測)

```
claude_code.interaction      (ユーザの1ターン全体)
├── claude_code.llm_request   (API 呼び出し。複数回あり得る)
├── claude_code.tool         (ツール実行)
│   ├── claude_code.tool.blocked_on_user (権限承認の待ち時間)
│   └── claude_code.tool.execution (ツール本体の実行時間)
└── claude_code.llm_request
```

各 span は所要時間をもつため、1ターンの時間を「API 待ち / ツール実行 / 権限待ち」へ分解でき、どこが遅いか(ボトルネック)を特定できる。**llm_request** span は **model / stop_reason / ttft_ms / tokens / request_id** 等を保持する。

出典: 公式ドキュメント「トレース(ベータ)」および span 階層・span 属性の各節
claude.com/docs/ja/monitoring-usage#traces-beta / [#span-hierarchy](#) / [#span-attributes](#)

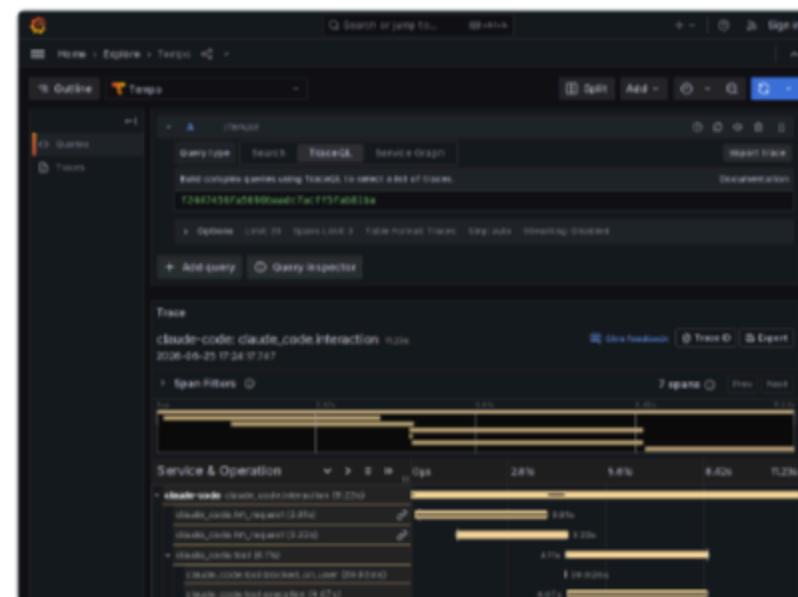


図4. Tempo のトレース表示(実測)。左に span 階層、右に各 span を開始時刻順に並べた横棒 (棒の長さ=所要時間)。これにより処理内訳が一目で分かる。

取得可能性の総括

監査したい情報	取得	条件
いつ・誰が(user/session/org)・量(token/cost)	可	既定
どのツールを・成否・所要時間・権限判断	可	既定
ユーザ入力 本文	可	OTEL_LOG_USER_PROMPTS
ツール入力引数 / 入出力内容	可	OTEL_LOG_TOOL_DETAILS / _CONTENT
モデル出力本文・会話履歴全文	可	OTEL_LOG_RAW_API_BODIES
分散トレース(処理の流れ・所要時間)	可	ENHANCED_TELEMETRY_BETA
拡張思考 (extended thinking) 本文	不可	常にマスク(構造的制約)

拡張思考(extended thinking)とは、モデルが回答を出す前に行う内部推論(思考)のテキストを指す。会話本文を取得できる唯一の経路は raw body 出力(`OTEL_LOG_RAW_API_BODIES`)だが、公式ドキュメントでは `api_response_body` は「拡張思考コンテンツはマスクされます」、`api_request_body` も「前のアシスタントターンの拡張思考コンテンツはマスクされます」と明記している。すなわち入力側・出力側の双方で拡張思考だけは除外され、テレメトリ経由では取得できない。

結論: テレメトリ単独ではほぼ全ての監査情報を取得・相関できる。取得できないのは拡張思考本文のみで、これも含めた会話の完全保全が必要な場合はローカルのセッションログ `~/.claude/projects/**/*.*.jsonl` を併用できる。

出典: 公式ドキュメント「API リクエストボディイベント」「API レスポンスボディイベント」
code.claude.com/docs/ja/monitoring-usage#api-request-body-event / [#api-response-body-event](https://code.claude.com/docs/ja/monitoring-usage#api-response-body-event)

考察

- **監査要件はテレメトリ単独でほぼ満たせる (§ 結果より)**。入力・出力・ツール入出力・処理の流れまで取得・相関でき、構造的に取得できないのは拡張思考本文のみであった。
- **取得範囲と機微性はトレードオフ (§ 5・§ 結果より)**。取得を広げるほど、会話本文・bash コマンド・ファイル内容など収集データ自体の機微性が高まる。
- **ゆえに取得範囲は目的に応じて段階的に絞るべき (§ 5 より)**。取得粒度は環境変数で制御できるため、監査目的に必要な最小限のみを有効化することが妥当である。
- **収集データの保護設計が前提となる**。会話本文等を含みうるため、保存先・保持期間・アクセス制御の設計が必要。本検証でも会話全文・生ログは外部公開の対象から除外した (§ 検証方法・成果物の扱い)。

まとめ

- Laptop と Docker Compose のみで、外部にデータを出さない監査基盤を構築できた。
- Claude Code の OpenTelemetry 出力**単独で、ユーザ入力・モデル出力・ツール入出力・分散トレースまで取得・**相関できることを実機で確認した。
- 取得粒度は環境変数で段階的に制御でき、構造的に取得できないのは**拡張思考本文のみ**であった。
- 取得範囲を広げるほど収集データの機微性が高まるため、運用では **目的に応じた最小限の有効化と収集データの保護設計**が前提となる。

1. Anthropic. 「Claude Code モニタリング (OpenTelemetry)」 .
<https://code.claude.com/docs/ja/monitoring-usage>
主な参照箇所(アンカー): #administrator-configuration(管理者設定)、#common-configuration-variables(環境変数)、#metrics・#metric-details(メトリクス)、#events ほか各イベント定義、#standard-attributes(標準属性)、#traces-beta・#span-hierarchy・#span-attributes(トレース)、#api-response-body-event(拡張思考のマスク)。
2. OpenTelemetry. 「OTLP Exporter Configuration」 . <https://opentelemetry.io/docs/specs/otel/protocol/exporter/>
3. Grafana Labs. Loki(OTLP ingestion)/ Tempo / grafana-image-renderer 各公式ドキュメント . <https://grafana.com/docs/>

付録: 本検証で確認したコンポーネント・バージョン

Claude Code 2.1.179 / otel-collector-contrib 0.115.1 / Prometheus 2.55.1 / Loki 3.3.2 / Tempo 2.6.1 / Grafana 11.4.0.

ライトニング版スライド: [presentation/index-lightning.html](#) / GUI 各画面の詳細解説: [presentation/gui-guide.md](#) .

